Teammate 2 (P2): architecture, MCP and Claude integration, enforcement

Slides 4 and 5. 2 minutes. P1 hands to you with: "So how can step three be guaranteed, when the model already obeyed?"

What you need to know

Product	Provenant: a deterministic firewall between AI agents and their tools. Every tool call gets ALLOW, DENY or REQUIRE_APPROVAL. The model is never the security boundary.
Team	P1 problem and use case (slides 1-3) · P2 architecture and enforcement (4-5) · P3 mithun: live demo, evaluation, limits, close (6-8). 6.5 minutes total.
Numbers you may quote	231/231 attacks contained · 53/53 legitimate actions allowed · 11/11 live proxy checks · 257 unit tests · decision p50 about 0.2 ms · 41 gaps found and fixed in red-team round 2.
What is covered	Only tool calls that go through a wrapped MCP server, the Claude Code hook or provenant serve. Remote HTTP MCP servers and agents calling APIs from their own code are not seen. Seeing a call is not stopping it: we only claim what we stop.
Never say	Unhackable, 100% secure, stops all prompt injection, tamper-proof, protects every app. Phone approval for held calls is built but not tested with a real call.
If stuck	"Great question, [owner] built that part." Hand over; never guess. Answers: team/QA_BANK.md.
Your job	Make the judges believe step three is guaranteed: one deterministic gate, three doors, fail closed, and an honest line on what is not covered. Then hand to mithun at the end of slide 5.

Slide 4: The model is never the security boundary (0:30)

> Because the decision isn't made by a model. Every tool call the agent proposes goes through one deterministic check, written in plain Python, with no AI in it. It returns one of three answers.
>
> ALLOW: it runs once, under a single-use grant tied to that exact call. DENY: it never reaches the tool. REQUIRE APPROVAL: it waits for a person.
>
> And it fails closed. If the policy file is broken or anything throws, the answer is DENY. A tool it has never seen waits for a person.

Slide 4 (continued): One gate, every entry point (0:35)

> There are three ways in, all on the left. MCP clients like Claude Desktop, Cursor and VS Code. Claude Code, through its hooks. And our own gated tool server.
>
> They all call the same function, Guard dot check. On the right: real tools only run after ALLOW plus a grant. Held calls sit in a small database with an expiry and can be used once. Every decision is written to an audit log where each record is chained to the previous one by hash and signed, so edits show up.
>
> What the agent reads goes back through the gate, so reading a malicious email changes what the agent is allowed to do next.
>
> And the line at the bottom is the honest part. Only calls that go through these three doors are protected. If an agent calls an API from its own code, or uses a remote MCP server, we don't see it, and provenant status won't say "connected" until real tool traffic has gone through us. Seeing a call is not the same as stopping it; we only claim what we stop.

Slide 5, right side: Drops into the agents people already use (0:30)

> No code changes for the user. provenant setup shows a plan, backs up each config, asks, then wraps every MCP server. Only allowed calls reach the server. We also pin each tool's definition. If a server changes a tool's description after you approved it, a "rug pull", the new definition is withheld.
>
> For Claude Code we use its hooks. DENY blocks the call, and REQUIRE APPROVAL becomes Claude's own "ask the human" prompt. We ran real Claude Code with permission checks switched off, told it to delete everything and read .env, and the hook blocked both.

Slide 5, left side: Six deterministic checks (0:25)

> Six independent checks. I'll name two. Provenance: an address the agent copied from something it read can't receive your data. That's exactly why the phishing send is denied. And single-use grants: an approved call can't be replayed or swapped for different arguments.
>
> Enough slides. *(turn to mithun)* [mithun] will show it live.

If a judge asks you something

- "What if the attacker paraphrases the address?" Provenance compares values, so a reworded address can slip that one rule. The other rules still apply: after the agent reads injected text, every side effect needs a human. It's on our limits slide.
- "Why not use an LLM to judge?" An LLM judge can be prompt-injected by the same text. We have an optional model, but it can only make decisions stricter, never allow anything, and it's off by default.
- "How fast?" About 0.2 milliseconds per decision on our test machine. The proxy adds roughly 11 to 25 milliseconds per call, including writing the audit log.
- "Can the agent approve its own call, or switch you off?" The policy denies an agent running provenant approve, rollback, uninstall or unwrap, or touching our state folder. provenant approve also refuses to run without a real terminal. We call that a speed bump, not a proof: an agent that writes a script and runs it is judged on the run command.

- "Remote MCP servers?" Not yet. Only local stdio servers are proxied today, and setup lists remote ones as not protected.
- "How do you know a denied call never ran?" The tests run a real MCP server that really deletes files and sends to an outbox, and check the disk afterwards, not the log.
- "Which models does it work with?" Any. It never looks inside the model; it only sees tool calls.

Do not say

- "Uses AI to detect attacks." The deciding part has no AI in it.
- "Tamper-proof logs." They are tamper-evident: edits are detected, but deleting the end leaves a shorter valid chain unless the head is anchored.